

1. GenAI: Gemini, ChatGPT
2. Accessed both through google search query
3. Prompt used:

Write a C program that does the following: The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

4. Code analysis
 - a. Correctness:
 - i. Test execution: ChatGPT's code actually runs while Gemini's runs every now and then. ChatGPT also passes the "happy trail" but fails with anything else. Gemini barely passes the happy trail and fails with anything else
 - b. Execution time:
 - i. Gemini has good execution time with using heapify_up and heapify_down. Its big O is something like $O(N \log n)$. ChatGPT bottlenecks itself. When it uses higherPriority, it calls datevalue() which has to call sscanf() to compare emails which slows the whole thing down. Its big is something like $O(N * n \log n)$.
 - c. Space complexity:
 - i. Gemini allocates a dynamic heap space. Memory expanded to scale to the number of emails. ChatGPT allocates a massive static block of local stack space inside the main block (Email heap[1000]). This will fail silently if the email trace goes over 1000 commands.
 - d. Maintainability:
 - i. They both have pretty decent comments. Gemini uses some duplication in the trim function, which isn't the best. ChatGPT is much better since the code is more isolated (less duplication) and easier to follow than Gemini
5. I chose ChatGPT's code since it's easier to understand and has less errors with it than Gemini. It actually runs, there's less duplication in the code, and the fixes shouldn't be too bad.
6. What was fixed:
 - a. Execution time:
 - i. Old code called sscanf repeatedly which caused bottlenecking. I added long date_numeric in the email definition space. This parses the date integer string only once which speeds things up.
 - b. Correctness:
 - i. The old code had a loop that assumed formatting patterns were perfect. So if an incoming command omitted commas, the pointer returned a null reference value, which would crash. I added if (!firstComma) return 0; to skip bad configurations rather than crashing.
 - c. Space complexity:
 - i. I increased the date string buffer from 11 to 12 characters to give room for error when checking a 10 character string.
 - d. Maintainability:
 - i. I removed some global multi-assignment variables and added some more visual separation just so the code is easier to look at and follow.

Gemini code:

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>

#define MAX_SUBJECT 256
#define INITIAL_CAPACITY 10

typedef struct {
    char category[32];
    char subject[MAX_SUBJECT];
    char date_str[12];
    int priority_score;
    long date_score; // Format: YYYYMMDD for easy numeric comparison
} Email;

typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;

// Helper function to map category to numeric priority
int get_category_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

// Helper function to convert MM-DD-YYYY to a YYYYMMDD long integer
long parse_date(const char *date_str) {
    int month, day, year;
    sscanf(date_str, "%d-%d-%d", &month, &day, &year);
    return (long)year * 10000 + month * 100 + day;
}

// Initialize MaxHeap
MaxHeap* create_heap() {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->size = 0;
    heap->capacity = INITIAL_CAPACITY;
    heap->data = (Email*)malloc(sizeof(Email) * heap->capacity);
    return heap;
}

// Compare two emails based on core requirements
int compare_emails(Email a, Email b) {
    if (a.priority_score != b.priority_score) {

```

```

        return a.priority_score - b.priority_score;
    }
    // Tie-breaker: Newest email (higher date score) comes first
    return (a.date_score > b.date_score) ? 1 : ((a.date_score < b.date_score) ? -1 : 0);
}

void swap(Email *x, Email *y) {
    Email temp = *x;
    *x = *y;
    *y = temp;
}

// Heapify up when inserting
void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare_emails(heap->data[index], heap->data[parent]) > 0) {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

// Heapify down when extracting
void heapify_down(MaxHeap *heap, int index) {
    int max_idx = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < heap->size && compare_emails(heap->data[left], heap->data[max_idx]) > 0) {
        max_idx = left;
    }
    if (right < heap->size && compare_emails(heap->data[right], heap->data[max_idx]) > 0) {
        max_idx = right;
    }
    if (index != max_idx) {
        swap(&heap->data[index], &heap->data[max_idx]);
        heapify_down(heap, max_idx);
    }
}

// Insert email into priority queue
void insert_email(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, sizeof(Email) * heap->capacity);
    }
}

```

```

    heap->data[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

// Extract the highest priority email (READ command)
void extract_max(MaxHeap *heap) {
    if (heap->size <= 0) return;
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    heapify_down(heap, 0);
}

// Print the top email (NEXT command)
void peek_next(MaxHeap *heap) {
    if (heap->size <= 0) return;
    Email top = heap->data[0];
    printf("Next email:\n");
    printf("Sender: %s\n", top.category);
    printf("Subject: %s\n", top.subject);
    printf("Date: %s\n", top.date_str);
}

// Trim whitespace from string ends
void trim(char *str) {
    int len = strlen(str);
    while (len > 0 && (str[len - 1] == ' ' || str[len - 1] == '\t' || str[len - 1] == '\r' || str[len - 1] == '\n')) {
        str[--len] = '\0';
    }
    int start = 0;
    while (str[start] == ' ' || str[start] == '\t') {
        start++;
    }
    if (start > 0) {
        memmove(str, str + start, len - start + 1);
    }
}

int main() {
    MaxHeap *heap = create_heap();
    char line[512];

    // Read commands from standard input until EOF
    while (fgets(line, sizeof(line), stdin)) {
        // Strip trailing newlines
        line[strcspn(line, "\r\n")] = 0;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email email;

```

```

char *payload = line + 6;

char *cat_tok = strtok(payload, ",");
char *sub_tok = strtok(NULL, ",");
char *date_tok = strtok(NULL, ",");

if (cat_tok && sub_tok && date_tok) {
    trim(cat_tok);
    trim(sub_tok);
    trim(date_tok);

    strcpy(email.category, cat_tok);
    strcpy(email.subject, sub_tok);
    strcpy(email.date_str, date_tok);

    email.priority_score = get_category_priority(email.category);
    email.date_score = parse_date(email.date_str);

    insert_email(heap, email);
}
}
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap->size);
}
else if (strcmp(line, "NEXT") == 0) {
    peek_next(heap);
}
else if (strcmp(line, "READ") == 0) {
    extract_max(heap);
}
}

// Clean up allocated memory
free(heap->data);
free(heap);
return 0;
}

```

ChatGPT code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define SUBJECT_SIZE 500

/*
 * Each email has:
 *  sender category
 *  subject
 *  date
 *
 * Priority:
 *  Boss      = 5
 *  Subordinate = 4
 *  Peer      = 3
 *  ImportantPerson = 2
 *  OtherPerson  = 1
 *
 * If two emails have the same sender category,
 * the newer date has higher priority.
 */

typedef struct {
    char sender[50];
    char subject[SUBJECT_SIZE];
    char date[11];    // MM-DD-YYYY
    int priority;
} Email;

/*
 * MaxHeap implemented using an array.
 */

typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;

/* -----
 * Utility Functions
 * ----- */

/*
```

* Convert a sender category into its priority value.

*/

```
int getPriority(char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1;
}
```

/*

* Convert MM-DD-YYYY into an integer that can be compared.

*

* For example:

* 12-20-2024 -> 20241220

*/

```
int dateValue(char *date)
{
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}
```

/*

* Return 1 if email1 has higher priority than email2.

*

* Sender category is considered first.

* Newer date is considered second.

*/

```
int higherPriority(Email *email1, Email *email2)
{
    if (email1->priority > email2->priority)
        return 1;

    if (email1->priority < email2->priority)
        return 0;
}
```

/*

* Same sender category:

```

    * newest email gets priority.
    */
    if (dateValue(email1->date) > dateValue(email2->date))
        return 1;

    return 0;
}

/* -----
 * MaxHeap Functions
 * ----- */

/*
 * Initialize an empty heap.
 */
void initializeHeap(MaxHeap *heap)
{
    heap->size = 0;
}

/*
 * Swap two emails.
 */
void swap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/*
 * Move an item upward in the heap.
 */
void heapifyUp(MaxHeap *heap, int index)
{
    int parent;

    while (index > 0)
    {
        parent = (index - 1) / 2;

        if (higherPriority(&heap->heap[index],
                        &heap->heap[parent]))
        {
            swap(&heap->heap[index],
                &heap->heap[parent]);

            index = parent;
        }
    }
}

```

```

        else
        {
            break;
        }
    }
}

/*
 * Move an item downward in the heap.
 */
void heapifyDown(MaxHeap *heap, int index)
{
    int left;
    int right;
    int largest;

    while (1)
    {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < heap->size &&
            higherPriority(&heap->heap[left],
                          &heap->heap[largest]))
        {
            largest = left;
        }

        if (right < heap->size &&
            higherPriority(&heap->heap[right],
                          &heap->heap[largest]))
        {
            largest = right;
        }

        if (largest != index)
        {
            swap(&heap->heap[index],
                 &heap->heap[largest]);

            index = largest;
        }
        else
        {
            break;
        }
    }
}

```

```

/*
 * Insert an email into the MaxHeap.
 */
void insert(MaxHeap *heap, Email email)
{
    if (heap->size >= MAX_EMAILS)
    {
        printf("Heap is full.\n");
        return;
    }

    heap->heap[heap->size] = email;

    heapifyUp(heap, heap->size);

    heap->size++;
}

/*
 * Return the highest-priority email without removing it.
 */
Email *peek(MaxHeap *heap)
{
    if (heap->size == 0)
        return NULL;

    return &heap->heap[0];
}

/*
 * Remove the highest-priority email.
 */
Email removeMax(MaxHeap *heap)
{
    Email removed;

    /*
     * This function should only be called when
     * the heap is not empty.
     */
    removed = heap->heap[0];

    heap->size--;

    if (heap->size > 0)
    {
        heap->heap[0] = heap->heap[heap->size];
    }
}

```

```

        heapifyDown(heap, 0);
    }

    return removed;
}

/* -----
 * Email Functions
 * ----- */

/*
 * Parse an EMAIL command.
 *
 * Expected format:
 *
 * EMAIL <sender category>,<subject line>,<date>
 */
Email parseEmail(char *line)
{
    Email email;
    char *firstComma;
    char *secondComma;

    /*
     * Remove the "EMAIL " portion.
     */
    char *data = line + 6;

    firstComma = strchr(data, ',');

    /*
     * Find the second comma after the subject.
     */
    secondComma = strchr(firstComma + 1, ',');

    /*
     * Extract sender.
     */
    *firstComma = '\0';
    strcpy(email.sender, data);

    /*
     * Extract subject.
     */
    *secondComma = '\0';
    strcpy(email.subject, firstComma + 1);

    /*
     * Extract date.

```

```

    */
    strcpy(email.date, secondComma + 1);

    /*
    * Remove newline from date if present.
    */
    email.date[strcspn(email.date, "\r\n")] = '\0';

    /*
    * Assign sender priority.
    */
    email.priority = getPriority(email.sender);

    return email;
}

/*
* Display the next email without removing it.
*/
void displayNext(MaxHeap *heap)
{
    Email *email = peek(heap);

    if (email == NULL)
        return;

    printf("Next email: Sender: %s Subject: %s Date: %s\n",
        email->sender,
        email->subject,
        email->date);
}

/*
* Read/remove the highest-priority email.
*
* READ does not display anything.
*/
void readEmail(MaxHeap *heap)
{
    if (heap->size == 0)
        return;

    removeMax(heap);
}

/*
* Display the current number of unread emails.
*/

```

```

void displayCount(MaxHeap *heap)
{
    printf("There are %d emails to read.\n", heap->size);
}

/* -----
 * Main
 * ----- */

int main(int argc, char *argv[])
{
    MaxHeap emailQueue;
    char line[SUBJECT_SIZE + 100];

    initializeHeap(&emailQueue);

    /*
     * The input file is supplied as a command-line argument.
     *
     * Example:
     * ./email_priority emails.txt
     */
    if (argc != 2)
    {
        printf("Usage: %s <input file>\n", argv[0]);
        return 1;
    }

    FILE *file = fopen(argv[1], "r");

    if (file == NULL)
    {
        printf("Could not open input file.\n");
        return 1;
    }

    /*
     * Process every line in the input file.
     */
    while (fgets(line, sizeof(line), file) != NULL)
    {
        /*
         * Remove newline from the end of the line.
         */
        line[strcspn(line, "\r\n")] = '\0';

        /*
         * Ignore blank lines.

```

```

    */
    if (strlen(line) == 0)
        continue;

    /*
    * EMAIL command
    */
    if (strncmp(line, "EMAIL ", 6) == 0)
    {
        Email email = parseEmail(line);

        insert(&emailQueue, email);
    }

    /*
    * NEXT command
    */
    else if (strcmp(line, "NEXT") == 0)
    {
        displayNext(&emailQueue);
    }

    /*
    * READ command
    */
    else if (strcmp(line, "READ") == 0)
    {
        readEmail(&emailQueue);
    }

    /*
    * COUNT command
    */
    else if (strcmp(line, "COUNT") == 0)
    {
        displayCount(&emailQueue);
    }
}

fclose(file);

return 0;
}

```